

برنامه‌سازی پیشرفته

تمرین کامپیوتری شماره ۳

مدرس: رامتین خسروی



طراحان: ریحانه عین‌اللهی، مهدیس میرزایی، رضا

چهرقانی، صادق صمدی، سید حمید محمودی،

محمد عرفان دانایی، مهرداد لیویان، سینتا غنودی

مهلت تحویل: دوشنبه ۱۹ آبان ۱۴۰۴، ساعت ۲۳:۵۹

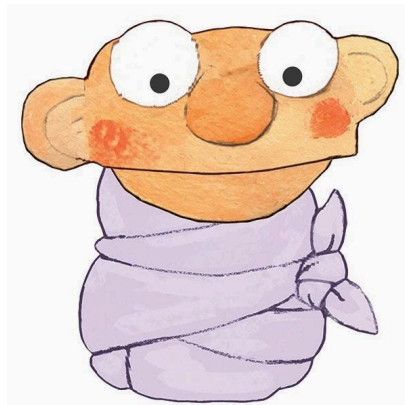
مقدمه

این تمرین برای آشنایی با برنامه‌نویسی بازگشتی طراحی و در قالب سه سوال مجزا تهیه شده است که پیشنهاد می‌شود برای درک بهتر مفاهیم برنامه‌نویسی بازگشتی، زمان کافی را برای پاسخ دادن به آن‌ها اختصاص دهید. توجه کنید که پرسش‌ها حتماً باید به روش بازگشتی حل شوند، هر چند ممکن است روش‌های دیگری نیز برای حل آن‌ها وجود داشته باشد.

قسمتی از نمره شما در این تمرین به طراحی آزمون‌های خودکار اختصاص داده شده است. شما باید برای هر سوال دو آزمون طراحی کنید. نحوه قراردعی و مکان این آزمون‌ها در قسمت نکات و نحوه تحویل توضیح داده شده است.

توجه کنید صحت آزمون‌های شما نیز بررسی می‌شود و اگر از نظر منطقی آزمون شما به درستی طراحی نشده باشد، نمره این بخش را دریافت نخواهید کرد.

تمرین اول: اسکندر در بازارهای مالی



اسکندر پس از تلاش‌های فراوان برای یادگیری علم اقتصاد متوجه شد در بازارهای مالی صرفاً باید تا می‌تواند محصولات ارزان بخرد. این راه بیشترین میزان سود را به او می‌دهد. شما برای اسکندر برنامه‌ای بنویسید که در آن F فروشنده داریم که هر یک محصولاتی دارند و قیمت این محصولات داده شده است. حال اسکندر در هر چرخه خرید مقداری پول وارد بازار می‌کند و ارزان‌ترین کالا را پیدا می‌کند. اگر قیمت آن از پول اسکندر کمتر یا مساوی باشد، اسکندر توانایی خرید دارد و آن را می‌خرد و قیمت کالا از پول ورودی اسکندر کم می‌شود و همچنین آن محصول از بازار حذف می‌شود و این چرخه با مقدار جدید پول اسکندر ادامه دارد. برای مثال اگر اسکندر ۱۰۰ تومان وارد بازار کند و قیمت ارزان‌ترین کالا ۲۰ تومان باشد، ادامه چرخه خرید با ۸۰ تومان است ولی اگر ارزان‌ترین کالا ۱۲۰ تومان باشد، چرخه خرید پایان می‌یابد.

توجه داشته باشید که استفاده از `for` و `while` در این سوال مجاز نمی‌باشد.

برای مرتب کردن آرایه می‌توانید از تابع آماده `sort` استفاده کنید. برای مطالعه بیشتر درباره آن به [لینک](#) می‌توانید مراجعه کنید.

ورودی

در خط اول عدد F داده می‌شود که نشان‌دهنده تعداد فروشنده‌ها است. در ادامه، F خط بعدی مربوط به اطلاعات هر فروشنده است. در هر یک از این خطوط، ابتدا یک عدد x می‌آید که تعداد محصولات آن فروشنده را مشخص می‌کند. سپس x عدد دیگر در همان خط نوشته می‌شود. x_i نشان‌دهنده قیمت محصولات آن فروشنده است. به عنوان مثال، اگر $x = 3$ باشد، در ادامه‌ی خط ۳ عدد (قیمت کالاها) می‌آید که لزومی ندارد به ترتیب خاصی باشند.

در خط بعد، عدد n داده می‌شود که نشان‌دهنده تعداد دفعاتی است که اسکندر قصد دارد یک چرخه‌ی خرید جدید ایجاد کند. سپس در n خط بعدی، برای هر چرخه، یک عدد n_i نوشته می‌شود. عدد n_i نشان می‌دهد که اسکندر در آغاز چرخه‌ی i -ام چه مقدار پول وارد بازار می‌کند.

دقت کنید که اگر از چرخه‌های قبل مقداری از پول باقی مانده بود، مقدار پول جدیدی که وارد این چرخه می‌کند با مقدار باقی‌مانده جمع می‌شود. فرض کنید قبل از شروع اولین چرخه پول اسکندر صفر می‌باشد.

$$1 \leq n \leq 10^2$$

$$1 \leq F \leq 10^2$$

$$2 \leq x \leq 10^2$$

$$2 \leq x_i \leq 10^2$$

$$1 \leq n_i \leq 10^2$$

توجه کنید پس از هر چرخه خرید، با کالاهای باقی‌مانده از چرخه قبلی ادامه می‌دهیم و به عبارتی کالاهای خریداری‌شده دیگر وجود ندارند و در حین چرخه نمی‌توان پول را تغییر داد یعنی اگر با ۱۰۰ تومان شروع کردیم هربار از آن مقداری کم می‌شود و وقتی دیگر نتوان چیزی خرید به پایان می‌رسد.

نمونه ورودی
2
2 10 20
3 80 9 2
2
19
50

توضیح ورودی

در خط اول تعداد فروشنده‌ها وارد شده است که برابر ۲ نفر است. در خط دوم و سوم به ترتیب کالاهای فروشنده اول و دوم وارد شده‌اند. فروشنده اول دو کالا به قیمت‌های ۱۰ و ۲۰ تومان دارد و فروشنده دوم سه کالا به قیمت‌های ۸۰ و ۹ و ۲ تومان دارد. در خط چهارم تعداد چرخه‌ها مشخص شده است که دو چرخه داریم. دو خط بعدی نشان‌دهنده مقدار پولی است که اسکندر در هر چرخه اضافه می‌کند. در چرخه اول ۱۹ تومان و در چرخه دوم ۵۰ تومان اضافه می‌شود.

خروجی

برای خروجی شما باید به ازای هر یک از n بار چرخه خرید، تعداد کالاها و قیمت کالاهای خریداری‌شده را در n خط نمایش دهید. قیمت کالاهای هر خط باید مرتب شده از کم به زیاد باشد.

در این مثال برای بار اول که ۱۹ تومان پول دارد ابتدا کالا ۲ تومانی را می‌خرد و سپس کالا ۹ تومانی را و دیگر چیزی نمی‌تواند بخرد. در بار دوم ابتدا کالا ۱۰ تومانی را می‌خرد و سپس کالای ۲۰ تومانی و دیگر نمی‌تواند چیزی بخرد.

نمونه خروجی		
2	2	9
2	10	20

تمرین دوم: اسکندر و رشته عجیب

اسکندر پس از ضررهای زیاد در بازارهای مالی متوجه شد باید از بازارهای مالی دوری کند. او ناگهان با رشته‌ای رو به رو شد که یک رشته عادی نبود. این رشته در ابتدا مقدار S داشت و هر بار به یکی از

aaS, caS, bS, aSa, cSb, aSbb, a, b, aa, cc, ba

تبدیل می‌شود. به عبارتی در ابتدا S است و سپس می‌تواند aaS شود و چون باز هم در آن S وجود دارد این مقدار S نیز می‌تواند مانند S ابتدایی تغییر کند و تبدیل شود ولی در نظر داشته باشید رشته‌ای که تولید می‌شود باقی رشته را تغییر نمی‌دهد و در کنار آن aa-ای که از تبدیل بار اول ایجاد شده بود باقی خواهد ماند.

S -> aaS -> aacaS -> aacaba

برای مثال در اینجا S به aaS تبدیل شد و S این رشته جدید به caS تبدیل شد و رشته به صورت aacaS شد و سپس S به ba تبدیل شد که باعث شد رشته نهایی aacaba شود و چرا نهایی است؟ زیرا دیگر در آن S-ای وجود ندارد که باعث تبدیل شدن آن به رشته‌های جدید شود. پس رشته‌های نهایی رشته‌هایی هستند که در آنها صرفاً حروف کوچک وجود دارد.

ورودی

در خط اول ورودی یک رشته می‌آید و شما باید مشخص کنید آیا با شروع از یک S تنها می‌توان به آن رسید یا خیر. پاسخ شما yes/no خواهد بود.

نمونه ورودی ۱
aacaba

خروجی

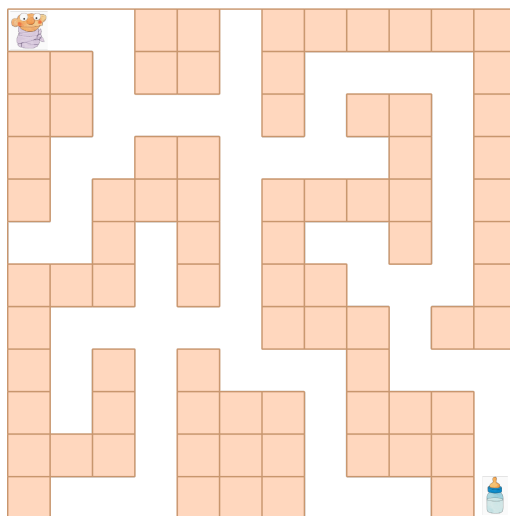
طبق مثال گفته شده در سوال پاسخ yes است.

نمونه خروجی ۱
yes

نمونه ورودی ۲
bc
نمونه خروجی ۲
no

صرفا a یا b را می‌توان با طول کمتر مساوی ۱ داشت.

تمرین سوم: اسکندر در مسیر شیشه شیر



اسکندر پس از حل مسئله‌ی رشته عجیب وارد یک هزارتوی بزرگ شد. در طول مسیر اسکندر به شدت تشنه شده بود و دنبال شیشه شیر خود می‌گشت که متوجه شد شیشه شیر در انتهای هزارتو قرار گرفته است. از آنجایی که اسکندر بسیار تشنه است، به او کمک کنید کوتاه‌ترین مسیر ممکن برای رسیدن به شیشه شیر را پیدا کند. اسکندر می‌تواند در هر یک از ۴ جهت بالا، پایین، چپ و راست در هر مرحله حرکت کند.

ورودی

در خط اول ورودی ابعاد نقشه که با n, m مشخص می‌شود می‌آید. n تعداد سطرها و m تعداد ستون‌های هزارتو است. سپس نقشه هزارتو در ورودی داده می‌شود. دیوارها (موانع) با عدد 1، مسیرها با عدد 0، اسکندر با حرف S و شیشه شیر با حرف M در نقشه مشخص می‌شوند.

نمونه ورودی ۱

```
6 9
S11001000
010010010
001100110
001000000
101101000
00000010M
```

خروجی

خروجی باید رشته‌ای از کاراکترهای `r`, `l`, `u`, `d` باشد که کوتاه‌ترین مسیر رسیدن از `S` به `M` را نشان می‌دهد. اگر چند مسیر با طول برابر وجود داشت شما باید فقط یکی از آن‌ها را خروجی دهید. (تمامی جواب‌های ممکن با طول حداقل، در آزمون‌های خودکار پذیرفته خواهند شد). اگر هیچ مسیری به شیشه شیر پیدا نشد، خروجی باید یک رشته‌ی `NO_PATH` باشد.

نمونه خروجی ۱
dddrddrrruurdrdr

نمونه ورودی ۲
6 8 S0100001 00101101 01100000 00011101 11000100 1111000M
نمونه خروجی ۲
dddrdrdrdr

نکات و نحوه تحویل

- برنامه شما باید در سیستم عامل لینوکس و با مترجم g++ با استاندارد c++20 ترجمه و در زمان معقول برای ورودی‌های آزمون اجرا شود.
- تحویل این تمرین در سامانه گیت‌هاب انجام می‌شود. برای انجام تمرین لطفاً از طریق [این لینک](#) وارد شوید، پس از آن باید شماره دانشجویی خود را انتخاب کنید (دقت کنید که با کمک این شماره دانشجویی به شما نمره خواهیم داد، لطفاً در انتخاب درست شماره دانشجویی حتماً دقت کنید، در صورتی که به مشکل خوردید با دستیاران ارشد در ارتباط باشید). پس از آن به صفحه‌ای منتقل می‌شوید که در آنجا می‌توانید تمرین جدید را قبول کنید، پس از قبول کردن تمرین یک مخزن¹ در [AP Assignments](#) برای شما ساخته می‌شود و باید کدهای خود را در آنجا قرار دهید.
- کد هر سوال را در یک فایل مجزا با فرمت Q#.cpp قرار دهید. برای مثال نام فایل حاوی کد پاسخ سوال ۱ می‌شود Q1.cpp. سپس فایل‌های کدهای خود را در پوشه src در مخزن ساخته شده قرار دهید. توجه کنید که فایل‌های کدهای شما باید در پوشه src مخزن باشند و نباید در مکان دیگری قرار داده شوند.
- آزمون هر سوال باید در پوشه tests مخزن شما و پوشه مربوط به هر سوال قرار بگیرند. فایل‌های مربوط به آزمون‌ها برای شما قرار گرفته و شما باید آزمون‌های خود را در این فایل‌ها قرار دهید. توجه کنید که فایل‌های آزمون‌های شما باید در پوشه‌های مشخص شده در مخزن باشند و نباید در مکان دیگری قرار داده شوند. برای مثال ورودی مربوط به آزمون اول برای سوال اول باید در فایل tests/Q1/01/01.in و خروجی مورد انتظار آن باید در فایل tests/Q1/01/01.out قرار داده شود.
- دقت داشته باشید صحت آزمون‌های طراحی شده توسط شما نسبت به موارد خواسته شده در صورت تمرین سنجیده می‌شود. بنابراین دقت کافی در طراحی درست آزمون‌ها داشته باشید. در صورتی که آزمون طراحی شده شما از نظر منطقی درست نباشد، نمره مربوط را از دست خواهید داد.
- سعی کنید مرحله به مرحله کد را کامیت کنید و از کامیت کردن کل کد در انتهای پروژه خودداری کنید. همچنین، پیام کامیت‌های شما باید خوانا باشد و تغییر ایجاد شده را توضیح دهد. دقت کنید که بخشی از نمره شما را استفاده صحیح از گیت تشکیل می‌دهد. برای مطالعه در خصوص نحوه صحیح کامیت کردن، می‌توانید [این لینک](#) را مطالعه کنید.

¹ Repository

- پس از انجام تمرین و بارگذاری در گیت‌هاب، کد Hash آخرین کامیت² را به همراه شماره دانشجویی خود در سامانه ای‌لرن آپلود کنید (در خط اول شماره دانشجویی، پس از آن از Enter استفاده کنید و به خط بعد بروید و پس از آن Hash آخرین کامیت). نمونه متن خواسته شده در سامانه ای‌لرن (بخش <last_commit_hash> و <sid> را جایگزین کنید):

```
<sid>
<last_commit_hash>
```

نمونه:

```
810100000
bad8fbcdcf3b9feb371a31e0c370150aa870b18
```

- دقت کنید که عدم رعایت ساختار گفته شده در آپلود یا تغییر ساختار فایل‌ها در مخزن (می‌توانید به دلخواه خود فایل اضافه کنید و ... اما اسم و ساختار فایل‌هایی که در ابتدا به شما داده می‌شود نباید تغییر کند) باعث کسر ۱۰ درصد از نمره شما خواهد شد.
- بخش مهمی از ارزیابی برنامه شما درستی عملکرد آن است. بنابراین به اندازه کافی ورودی‌های آزمایشی طراحی کنید تا درستی خروجی در حالت‌های مختلف آزموده شود و دقت کنید که اگر برنامه شما با مثال‌های داده شده درست کار کند لزوماً به معنای درستی در تمام سناریوها نیست.
- درستی برنامه شما از طریق آزمون‌های خودکار سنجیده می‌شود. به این ترتیب، لازم است خروجی تولید شده از نظر بزرگی و کوچکی حروف، رعایت فاصله‌ها، عدم وجود خروجی‌های اضافه، ... دقیقاً مانند نمونه‌های داده شده باشد. بنابراین پیشنهاد می‌شود که با استفاده از ابزارهایی مانند diff فرمت خروجی برنامه خود را با خروجی‌هایی که در اختیارتان قرار داده شده است مطابقت دهید.
- هدف این تمرین یادگیری شماست. لطفاً تمرین را خودتان انجام دهید. در صورت کشف تقلب مطابق سیاست درس با آن برخورد خواهد شد.

² Commit

نمرات

- تمیزی کد

- رعایت کردن نام‌گذاری صحیح و انسجام³
- عدم وجود کد تکراری
- رعایت دندانه‌گذاری⁴
- استفاده صحیح از متغیرهای ثابت به جای Magic Value-ها
- ساختاردهی کد در قالب توابع کوتاه که فقط یک کار را انجام می‌دهند

- درستی کد

- آزمون‌های خودکار

- طراحی بازگشتی

- پیاده‌سازی الگوریتم بازگشتی و عدم استفاده از حلقه (به جز موارد مجاز)

- گیت و گیت‌هاب

- استفاده از commit message-های مناسب
- هر کامیت یک کار مشخص انجام بدهد و کامیتی چندین کار انجام ندهد.

- طراحی آزمون‌ها

- بررسی حالات مختلف
- صحت آزمون

دقت کنید که موارد ذکر شده لزوماً کل نمره شما را تشکیل نمی‌دهند و ممکن است با تغییراتی همراه باشند.

³ Consistency

⁴ Indentation